

Trellis

A persistent memory graph for coding agents. Markdown on disk, a typed graph over it, and one MCP server that serves Claude Code, Codex, Cursor, and anything else that speaks the protocol.

No server

No API keys

TypeScript / Node 20+

MCP-first

Obsidian-readable vault

Codename is a placeholder – swap it freely.

WHAT WE'RE BUILDING

Three layers, and only one of them is the product

The VS Code extension is the shell, not the pipe. The MCP server is what actually delivers the value, which means Trellis works in a bare Codex terminal with no editor involved at all. Build in this order and the extension becomes optional polish rather than a dependency.

LAYER 1 – CORE	@trellis/core – npm package Vault read/write, the typed graph, the index, and retrieval. Knows nothing about agents or editors. Pure functions over a folder of markdown, which makes it trivially testable and the only place the hard logic lives.
LAYER 2 – ADAPTER	@trellis/mcp – stdio MCP server Exposes <code>memory_search</code>, <code>memory_expand</code>, <code>memory_write</code>, <code>memory_link</code>. One binary, every client. This is the product — it ships to npm and runs via <code>npm</code> with no install step.
LAYER 3 – SHELL	VS Code extension One-click wiring into every installed agent, the graph view, the write review queue, and the token-budget meter. Adds visibility and setup convenience. Removing it degrades nothing.

THE HARD PROBLEM

Making the agent actually write back

You've ruled out a separate extraction model, so the agent writes its own memory. That's the right call for cost and friction, but it inherits the known failure mode: agents deprioritize bookkeeping when they're mid-task. Nobody has solved this well, so solving it is the product.

The answer is redundancy — five independent channels, ordered from universal to deep. Every client gets at least the first three, so reliability degrades gracefully instead of collapsing on clients without hooks.

01 Tool descriptions as instruction surface

The MCP tool description is the one piece of your text that *every* agent loads, every session, unconditionally. Treat it as prompt engineering, not documentation: state when to write, what qualifies, and what format. Version it and measure it.
Reach: 100% of MCP clients

02 Live state in every tool response

Append a compact footer to each `memory_search` result: `[3 decisions observed, 0 written · call memory_write before you finish]`. This is the only *dynamic* channel that reaches every client — it re-prompts at exactly the moment the agent is already reading your output.
Reach: 100% of MCP clients · highest leverage

03 Generated rules blocks

Write a ~15-line managed block into `CLAUDE.md`, `AGENTS.md`, `.cursor/rules/`, `.windsurfrules`. Delimit it with markers so regeneration never clobbers the user's own content. Static, but it front-loads the protocol before the first turn.
Reach: most editors and CLIs

04 Deterministic hooks where they exist

Claude Code's `Stop` and `PreCompact` hooks let you check "did `memory_write` fire this session?" and, if not, block once with a directive. This costs nothing extra — it's the agent's own inference, forced at the right moment. The strongest rung, available on the fewest clients.
Reach: Claude Code · partial on Codex

05 Zero-LLM session journal

Passively record files touched, commands run, and tests failed. This is free and always happens. It gives the agent raw material to summarize instead of asking it to recall a whole session from memory — and it's a usable fallback record even when every rung above fails.
Reach: universal · no model involved

RETRIEVAL

Hybrid seed, graph expansion, hard budget

Pure graph traversal can't answer "what matters for this prompt," and pure vector search can't answer "what else is connected." Run both, in that order, and cap the output.

SEED	BM25 over node titles and summaries picks 3–5 entry points. Optional local vectors add semantic recall in Phase 3 — BM25 alone is already strong on code identifiers.
EXPAND	Walk 1–2 hops along typed edges from each seed. <code>supersedes</code> edges are followed to the current node and the superseded one is dropped, never returned.
RANK	Score by relevance × recency × confidence, penalized by hop distance. Deduplicate by node identity.
BUDGET	Truncate to a hard ceiling — default 1,500 tokens. Return skeletons only: id, title, one-line summary. Roughly 15 tokens per node, so ~40 nodes of map for the price of two files.
EXPAND ON DEMAND	<code>memory_expand(id)</code> returns full bodies. The agent spends tokens only on what it decided it needed, which is the entire savings mechanism.

DATA MODEL

Typed nodes, typed edges, enforced

Freeform graphs rot into noise within a week. Constrain the vocabulary at the tool boundary — `memory_write` rejects a node without a valid type, so the schema is enforced by the API rather than by hope.

NODE TYPE	HOLDS	EXAMPLE
Decision	A choice made and its reasoning	Chose BM25 over embeddings for v1 — no native deps
Constraint	A rule the work must respect	Vault must stay readable without the extension
Component	A part of the system and its role	The retrieval ranker in core/rank.ts
Entity	Domain vocabulary the agent keeps re-learning	What "rung" means in this codebase
Preference	How the user wants work done	No emoji in commit messages
Gotcha	A failure and what fixed it	sql.js ships without FTS5 — don't rely on it

EDGE supersedes The one everyone skips and regrets. Without it, an agent confidently acts on a decision you reversed weeks ago — worse than having no memory at all.	EDGE contradicts Detected on write by comparing against near-duplicates. Surfaces a conflict in the review queue instead of silently storing both.
EDGE depends_on / implements Structural links that make one-hop expansion meaningful. This is what a vector store cannot give you.	EDGE observed_in Provenance back to the session that produced the fact. Cheap to store, essential for auditing why something is in the graph.

Every node carries `confidence` and `last_confirmed` in frontmatter. Confidence decays with age unless re-confirmed by retrieval, which keeps stale facts from outranking fresh ones without ever deleting anything.

STACK

Chosen for zero install failures

CONCERN	CHOICE	WHY
Language	TypeScript, Node 20+	Same runtime as the extension host and every MCP client
Protocol	@modelcontextprotocol/sdk	The one interface all target agents share
Storage	Markdown + YAML frontmatter	Source of truth. Git-diffable, agent-readable, opens in Obsidian
Links	<code>[[wikilink]]</code> syntax	Obsidian-compatible for free; the graph is inspectable without us
Index	Pure-TS inverted index + BM25	No SQLite, no WASM, no prebuilds, no platform matrix. ~150 lines, instant on thousands of notes, and rebuildable from the vault at any time

CONCERN	CHOICE	WHY
Embeddings	transformers.js + bge-small ONNX	Phase 3, opt-in, lazy-downloaded. Local CPU, no key, works offline once cached
Graph view	Cytoscape.js in a webview	Handles thousands of nodes; no Obsidian app required
Bundling	esbuild + vsce	Single-file output keeps the VSIX small and startup fast
Testing	Vitest + a retrieval eval set	The eval set is the differentiator — see below
Distribution	npm, VS Code Marketplace, Open VSX	Open VSX is what reaches Cursor and Windsurf users

The index choice is the important one. Dropping SQLite removes native prebuilds for five platforms — historically the single largest source of "extension won't install" reports. Revisit only if a real vault gets slow, and the markdown stays authoritative either way, so migration is a re-index rather than a data migration.

ROADMAP

Five phases, each with a gate

Every phase ends in something you can use yourself. Don't advance past a gate that hasn't been met — especially Phase 1's, which is the one that decides whether this project is worth finishing.

00

WEEK 1

Spike – prove the loop closes

- Define the vault format: frontmatter fields, node types, wikilinks
- MCP server with three tools, naive substring search, no index
- Wire into Claude Code and Codex by hand; confirm both can read and write
- Dogfood on your CRM and Ovetra projects

GATE

An agent in one session writes a fact that an agent in a different session and a different client retrieves unprompted.

01

WEEKS 2–3

Retrieval core – the make-or-break phase

- BM25 index, incremental rebuild on file change
- Graph expansion, ranking, hard token budget
- Two-tier skeleton / expand-on-demand split
- Build the eval harness: 50 real questions from your own transcripts, each with a known-correct answer set

GATE

Budgeted retrieval beats plain repo grep on the eval set. If it doesn't, stop and fix retrieval – no amount of UI rescues this.

02

WEEK 4

Write-back ladder

- Tool-description iteration, measured against write rate
- Response footers with live session state
- Rules-file generator with managed delimiters
- Claude Code `Stop` / `PreCompact` hooks
- Session journal, and dedup-on-write with contradiction detection

GATE

Across 20 unattended sessions spanning three clients, $\geq 80\%$ produce at least one useful memory write with no human nudge.

03

WEEKS 5–6

VS Code extension

- Auto-detect installed agents; write their configs on one click
- Cytoscape graph view with type filters and click-to-open
- Review queue for proposed writes, conflicts, and contradictions
- Token-budget meter: injected vs. saved, per session
- Optional local embeddings behind a setting

GATE

Clean install on a fresh machine wires up every installed agent with zero terminal commands.

04

WEEKS 7–8

Harden and publish

- Compatibility matrix: Claude Code, Codex, Cursor, Windsurf, Zed, Continue
- Vault migration and versioning; corruption recovery via full re-index
- Docs, a 60-second demo, and published eval numbers
- Ship to npm, VS Code Marketplace, and Open VSX

GATE

Someone who has never seen the project installs it and gets a useful retrieval within five minutes.

POSITIONING

Five things nobody else is doing

This space already has basic-memory, mem0, OpenMemory, Cipher, and claude-mem.

"Markdown memory for agents" is not a differentiator on its own. These are.

01 Budget as a contract Retrieval never exceeds a stated token ceiling, and the meter shows what was spent versus saved. Everyone claims token savings; nobody proves them.	02 Memory that self-corrects supersedes , contradiction detection, and confidence decay. Every competitor appends forever and slowly poisons its own context.
03 Write-back that survives missing hooks Five redundant channels means it works in Cursor and Zed, not just where a hook API happens to exist.	04 A published eval harness Real numbers against a real baseline. It's the only credible answer to "does this actually help," and it doubles as your regression suite.
05 Nothing to trust No account, no key, no server, no telemetry. Memory of a proprietary codebase never leaves the disk — and the files stay readable if the project is abandoned.	

RISKS

What actually kills this

Write-back reliability The single biggest threat. If agents skip writing, the graph stays empty and nothing downstream matters. This is why Phase 2 has a measured gate rather than a subjective one.
Graph rot Without aggressive dedup on write, the vault doubles every few sessions and retrieval precision collapses. Dedup is not a Phase 4 nicety — it ships with the first write path.
Client drift Each agent handles MCP tool results, config paths, and rules files slightly differently, and all of them change often. Keep the compatibility matrix in CI and expect ongoing maintenance.
Building the UI too early The graph view is the fun part and contributes nothing until retrieval is genuinely good. Phase 3 sits after Phase 1's gate deliberately.